Show that the second smallest of n elements can be found with $n + \lceil \lg n \rceil - 2$ comparisons in the worst case. (*Hint:* Also find the smallest element.)

9.1-2

Given $n > 2$ distinct numbers, you want to find a number that is neither the minimum nor the maximum. What is the smallest number of comparisons that you need to perform?

9.1-3

A racetrack can run races with five horses at a time to determine their relative speeds. For 25 horses, it takes six races to determine the fastest horse, assuming transitivity (see page 1159). What's the minimum number of races it takes to determine the fastest three horses out of 25?

★ ***9.1-4***

Prove the lower bound of $\lceil 3n/2 \rceil - 2$ comparisons in the worst case to find both the maximum and minimum of n numbers. (*Hint:* Consider how many numbers are potentially either the maximum or minimum, and investigate how a comparison affects these counts.)

9.2 Selection in expected linear time

The general selection problem—finding the ith order statistic for any value of i—appears more difficult than the simple problem of finding a minimum. Yet, surprisingly, the asymptotic running time for both problems is the same: $\Theta(n)$. This section presents a divide-and-conquer algorithm for the selection problem. The algorithm RANDOMIZED-SELECT is modeled after the quicksort algorithm of Chapter 7. Like quicksort it partitions the input array recursively. But unlike quicksort, which recursively processes both sides of the partition, RANDOMIZED-SELECT works on only one side of the partition. This difference shows up in the analysis: whereas quicksort has an expected running time of $\Theta(n \lg n)$, the expected running time of RANDOMIZED-SELECT is $\Theta(n)$, assuming that the elements are distinct.

RANDOMIZED-SELECT uses the procedure RANDOMIZED-PARTITION introduced in Section 7.3. Like RANDOMIZED-QUICKSORT, it is a randomized algorithm, since its behavior is determined in part by the output of a random-number generator. The RANDOMIZED-SELECT procedure returns the ith smallest element of the array $A[p : r]$, where $1 \le i \le r - p + 1$.

```
RANDOMIZED-SELECT(A, p, r, i)
1 if p == r
2     return A[p] // 1 ≤ i ≤ r – p + 1 when p == r means that i = 1
3 q = RANDOMIZED-PARTITION(A, p, r)
4 k = q – p + 1
5 if i == k
6     return A[q] // the pivot value is the answer
7 elseif i < k
8     return RANDOMIZED-SELECT(A, p, q – 1, i)
9 else return RANDOMIZED-SELECT(A, q + 1, r, i – k)
```

Figure 9.1 illustrates how the RANDOMIZED-SELECT procedure works. Line 1 checks for the base case of the recursion, in which the subarray $A[p : r]$ consists of just one element. In this case, i must equal 1, and line 2 simply returns $A[p]$ as the ith smallest element. Otherwise, the call to RANDOMIZED-PARTITION in line 3 partitions the array $A[p : r]$ into two (possibly empty) subarrays $A[p : q - 1]$ and $A[q + 1 : r]$ such that each element of $A[p : q - 1]$ is less than or equal to $A[q]$, which in turn is less than each element of $A[q + 1 : r]$. (Although our analysis assumes that the elements are distinct, the procedure still yields the correct result even if equal elements are present.) As in quicksort, we'll refer to $A[q]$ as the ***pivot*** element. Line 4 computes the number k of elements in the subarray $A[p : q]$, that is, the number of elements in the low side of the partition, plus 1 for the pivot element. Line 5 then checks whether $A[q]$ is the ith smallest element. If it is, then line 6 returns $A[q]$. Otherwise, the algorithm determines in which of the two subarrays $A[p: q - 1]$ and $A[q + 1 : r]$ the ith smallest element lies. If $i < k$, then the desired element lies on the low side of the partition, and line

8 recursively selects it from the subarray. If $i > k$, however, then the desired element lies on the high side of the partition. Since we already know k values that are smaller than the ith smallest element of $A[p : r]$—namely, the elements of $A[p : q]$—the desired element is the $(i - k)$th smallest element of $A[q + 1 : r]$, which line 9 finds recursively. The code appears to allow recursive calls to subarrays with 0 elements, but Exercise 9.2-1 asks you to show that this situation cannot happen.

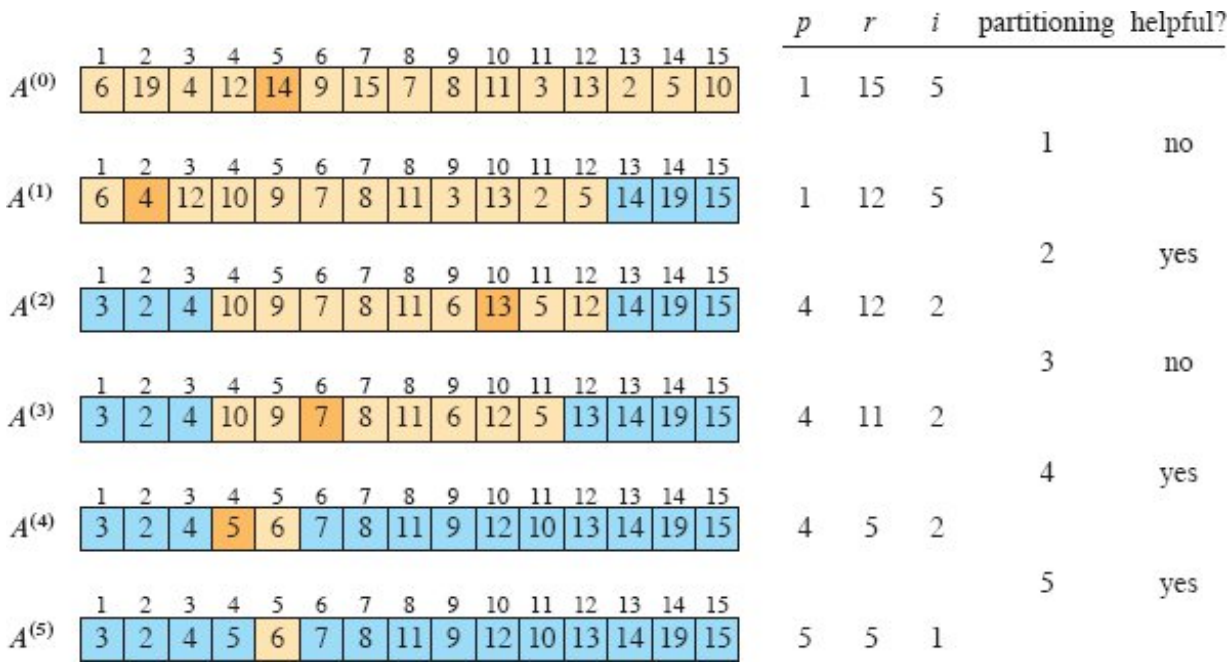

Figure 9.1 The action of RANDOMIZED-SELECT as successive partitionings narrow the subarray $A[p: r]$, showing the values of the parameters p, r, and i at each recursive call. The subarray $A[p : r]$ in each recursive step is shown in tan, with the dark tan element selected as the pivot for the next partitioning. Blue elements are outside $A[p : r]$. The answer is the tan element in the bottom array, where $p = r = 5$ and $i = 1$. The array designations $A^{(0)}$, $A^{(1)}$, … , $A^{(5)}$, the partitioning numbers, and whether the partitioning is helpful are explained on the following page.

The worst-case running time for RANDOMIZED-SELECT is $\Theta(n^2)$, even to find the minimum, because it could be extremely unlucky and always partition around the largest remaining element before identifying the ith smallest when only one element remains. In this worst case, each recursive step removes only the pivot from consideration. Because partitioning n elements takes $\Theta(n)$ time, the

recurrence for the worst-case running time is the same as for QUICKSORT: $T(n) = T(n-1) + \Theta(n)$, with the solution $T(n) = \Theta(n^2)$. We'll see that the algorithm has a linear expected running time, however, and because it is randomized, no particular input elicits the worst-case behavior.

To see the intuition behind the linear expected running time, suppose that each time the algorithm randomly selects a pivot element, the pivot lies somewhere within the second and third quartiles—the "middle half"—of the remaining elements in sorted order. If the ith smallest element is less than the pivot, then all the elements greater than the pivot are ignored in all future recursive calls. These ignored elements include at least the uppermost quartile, and possibly more. Likewise, if the ith smallest element is greater than the pivot, then all the elements less than the pivot—at least the first quartile—are ignored in all future recursive calls. Either way, therefore, at least 1/4 of the remaining elements are ignored in all future recursive calls, leaving at most 3/4 of the remaining elements ***in play***: residing in the subarray $A[p : r]$. Since RANDOMIZED-PARTITION takes $\Theta(n)$ time on a subarray of n elements, the recurrence for the worst-case running time is $T(n) = T(3n/4) + \Theta(n)$. By case 3 of the master method (Theorem 4.1 on page 102), this recurrence has solution $T(n) = \Theta(n)$.

Of course, the pivot does not necessarily fall into the middle half every time. Since the pivot is selected at random, the probability that it falls into the middle half is about 1/2 each time. We can view the process of selecting the pivot as a Bernoulli trial (see Section C.4) with success equating to the pivot residing in the middle half. Thus the expected number of trials needed for success is given by a geometric distribution: just two trials on average (equation (C.36) on page 1197). In other words, we expect that half of the partitionings reduce the number of elements still in play by at least 3/4 and that half of the partitionings do not help as much. Consequently, the expected number of partitionings at most doubles from the case when the pivot always falls into the middle half. The cost of each extra partitioning is less than the one that preceded it, so that the expected running time is still $\Theta(n)$.

To make the above argument rigorous, we start by defining the random variable $A^{(j)}$ as the set of elements of A that are still in play after j partitionings (that is, within the subarray $A[p : r]$ after j calls of RANDOMIZED-SELECT), so that $A^{(0)}$ consists of all the elements in A. Since each partitioning removes at least one element—the pivot—from being in play, the sequence $|A^{(0)}|$, $|A^{(1)}|$, $|A^{(2)}|$, … strictly decreases. Set $A^{(j-1)}$ is in play before the j th partitioning, and set $A^{(j)}$ remains in play afterward. For convenience, assume that the initial set $A^{(0)}$ is the result of a 0th "dummy" partitioning.

Let's call the j th partitioning ***helpful*** if $|A^{(j)}| \le (3/4)|A^{(j-1)}|$. Figure 9.1 shows the sets $A^{(j)}$ and whether partitionings are helpful for an example array. A helpful partitioning corresponds to a successful Bernoulli trial. The following lemma shows that a partitioning is at least as likely to be helpful as not.

Lemma 9.1
A partitioning is helpful with probability at least 1/2.

Proof Whether a partitioning is helpful depends on the randomly chosen pivot. We discussed the "middle half" in the informal argument above. Let's more precisely define the middle half of an n-element subarray as all but the smallest $\lceil n/4 \rceil - 1$ and greatest $\lceil n/4 \rceil - 1$ elements (that is, all but the first $\lceil n/4 \rceil - 1$ and last $\lceil n/4 \rceil - 1$ elements if the subarray were sorted). We'll prove that if the pivot falls into the middle half, then the pivot leads to a helpful partitioning, and we'll also prove that the probability of the pivot falling into the middle half is at least 1/2.

Regardless of where the pivot falls, either all the elements greater than it or all the elements less than it, along with the pivot itself, will no longer be in play after partitioning. If the pivot falls into the middle half, therefore, at least $\lceil n/4 \rceil - 1$ elements less than the pivot or $\lceil n/4 \rceil - 1$ elements greater than the pivot, plus the pivot, will no longer be in play after partitioning. That is, at least $\lceil n/4 \rceil$ elements will no longer be in play. The number of elements remaining in play will be at most $n - \lceil n/4 \rceil$,

which equals $\lfloor 3n/4 \rfloor$ by Exercise 3.3-2 on page 70. Since $\lfloor 3n/4 \rfloor \le 3n/4$, the partitioning is helpful.

To determine a lower bound on the probability that a randomly chosen pivot falls into the middle half, we determine an upper bound on the probability that it does not. That probability is

$$\begin{aligned}\frac{2(\lceil n/4 \rceil - 1)}{n} &\le \frac{2((n/4+1)-1)}{n} \quad \text{(by inequality (3.2) on page 64)} \\ &= \frac{n/2}{n} \\ &= 1/2 .\end{aligned}$$

Thus, the pivot has a probability of at least 1/2 of falling into the middle half, and so the probability is at least 1/2 that a partitioning is helpful.

■

We can now bound the expected running time of RANDOMIZED-SELECT.

Theorem 9.2

The procedure RANDOMIZED-SELECT on an input array of n distinct elements has an expected running time of $\Theta(n)$.

Proof Since not every partitioning is necessarily helpful, let's give each partitioning an index starting at 0 and denote by $\langle h_0, h_1, h_2, \ldots, h_m \rangle$ the sequence of partitionings that are helpful, so that the h_kth partitioning is helpful for $k = 0, 1, 2, \ldots, m$. Although the number m of helpful partitionings is a random variable, we can bound it, since after at most $\lceil \log_{4/3} n \rceil$ helpful partitionings, only one element remains in play. Consider the dummy 0th partitioning as helpful, so that $h_0 = 0$. Denote $|A^{(h_k)}|$ by n_k, where $n_0 = |A^{(0)}|$ is the original problem size. Since the h_kth partitioning is helpful and the sizes of the sets $A^{(j)}$ strictly decrease, we have $n_k = |A^{(h_k)}| \le (3/4)|A^{(h_k-1)}| = (3/4)\,n_{k-1}$ for $k = 1, 2, \ldots, m$. By iterating $n_k \le (3/4)\,n_{k-1}$, we have that $n_k \le (3/4)^k n_0$ for $k = 0, 1, 2, \ldots, m$.

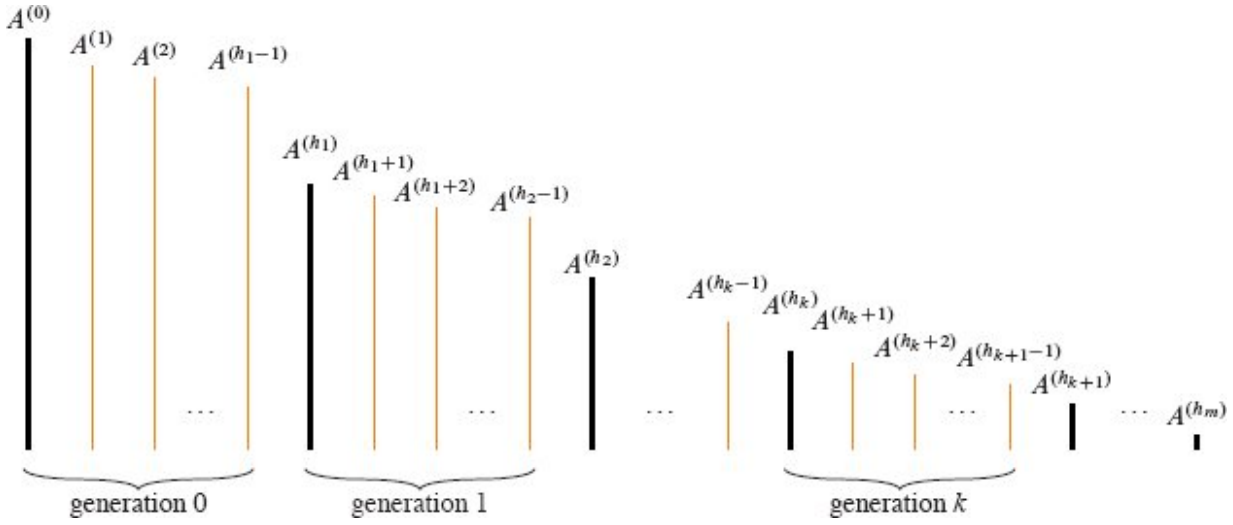

Figure 9.2 The sets within each generation in the proof of Theorem 9.2. Vertical lines represent the sets, with the height of each line indicating the size of the set, which equals the number of elements in play. Each generation starts with a set $A^{(h_k)}$, which is the result of a helpful partitioning. These sets are drawn in black and are at most 3/4 the size of the sets to their immediate left. Sets drawn in orange are not the first within a generation. A generation may contain just one set. The sets in generation k are $A^{(h_k)}, A^{(h_k)}, A^{(h_k+1)}, \ldots, A^{(h_{k+1}-1)}$. The sets $A^{(h_k)}$ are defined so that $|A^{(h_k)}| \le (3/4)|A^{(h_k-1)}|$. If the partitioning gets all the way to generation h_m, set $A^{(h_m)}$ has at most one element in play.

As Figure 9.2 depicts, we break up the sequence of sets $A^{(j)}$ into m ***generations*** consisting of consecutively partitioned sets, starting with the result $A^{(h_k)}$ of a helpful partitioning and ending with the last set $A^{(h_{k+1}-1)}$ before the next helpful partitioning, so that the sets in generation k are $A^{(h_k)}, A^{(h_k+1)}, \ldots, A^{(h_{k+1}-1)}$. Then for each set of elements $A^{(j)}$ in the kth generation, we have that $|A^{(j)}| \le |A^{(h_k)}| = n_k \le (3/4)^k n_0$.

Next, we define the random variable

$$X_k = h_k + 1 - h_k$$

for $k = 0, 1, 2, \ldots, m - 1$. That is, X_k is the number of sets in the kth generation, so that the sets in the kth generation are $A^{(h_k)}, A^{(h_k)}, A^{(h_k+1)}, \ldots, A^{(h_k+X_k-1)}$.

By Lemma 9.1, the probability that a partitioning is helpful is at least 1/2. The probability is actually even higher, since a partitioning is helpful even if the pivot does not fall into the middle half but the ith

smallest element happens to lie in the smaller side of the partitioning. We'll just use the lower bound of 1/2, however, and then equation (C.36) gives that $\mathrm{E}[X_k] \le 2$ for $k = 0, 1, 2, \ldots, m-1$.

Let's derive an upper bound on how many comparisons are made altogether during partitioning, since the running time is dominated by the comparisons. Since we are calculating an upper bound, assume that the recursion goes all the way until only one element remains in play. The j th partitioning takes the set $A^{(j-1)}$ of elements in play, and it compares the randomly chosen pivot with all the other $|A^{(j-1)}| - 1$ elements, so that the jth partitioning makes fewer than $|A^{(j-1)}|$ comparisons. The sets in the kth generation have sizes $|A^{(h_k)}|, |A^{(h_k+1)}|, \ldots, |A^{(h_k+X_k-1)}|$. Thus, the total number of comparisons during partitioning is less than

$$\begin{aligned}\sum_{k=0}^{m-1}\sum_{j=h_k}^{h_k+X_k-1}|A^{(j)}| &\le \sum_{k=0}^{m-1}\sum_{j=h_k}^{h_k+X_k-1}|A^{(h_k)}| \\ &= \sum_{k=0}^{m-1} X_k\,|A^{(h_k)}| \\ &\le \sum_{k=0}^{m-1} X_k \left(\frac{3}{4}\right)^k n_0 \,.\end{aligned}$$

Since $\mathrm{E}[X_k] \le 2$, we have that the expected total number of comparisons during partitioning is less than

$$
\begin{aligned}
\mathrm{E}\left[\sum_{k=0}^{m-1} X_k \left(\frac{3}{4}\right)^k n_0\right] &= \sum_{k=0}^{m-1} \mathrm{E}\left[X_k \left(\frac{3}{4}\right)^k n_0\right] \quad \text{(by linearity of expectation)} \\
&= n_0 \sum_{k=0}^{m-1} \left(\frac{3}{4}\right)^k \mathrm{E}[X_k] \\
&\le 2n_0 \sum_{k=0}^{m-1} \left(\frac{3}{4}\right)^k \\
&< 2n_0 \sum_{k=0}^{\infty} \left(\frac{3}{4}\right)^k \\
&= 8n_0 \qquad \text{(by equation (A.7) on page 1142)} .
\end{aligned}
$$

Since n_0 is the size of the original array A, we conclude that the expected number of comparisons, and thus the expected running time, for RANDOMIZED-SELECT is $O(n)$. All n elements are examined in the first call of RANDOMIZED-PARTITION, giving a lower bound of $\Omega(n)$. Hence the expected running time is $\Theta(n)$.

■

Exercises

9.2-1

Show that RANDOMIZED-SELECT never makes a recursive call to a 0-length array.

9.2-2

Write an iterative version of RANDOMIZED-SELECT.

9.2-3

Suppose that RANDOMIZED-SELECT is used to select the minimum element of the array $A = \langle 2, 3, 0, 5, 7, 9, 1, 8, 6, 4 \rangle$. Describe a sequence of partitions that results in a worst-case performance of RANDOMIZED-SELECT.

9.2-4

Argue that the expected running time of RANDOMIZED-SELECT does not depend on the order of the elements in its input array $A[p : r]$. That is, the expected running time is the same for any permutation of the input array $A[p : r]$. (*Hint:* Argue by induction on the length n of the input array.)

9.3 Selection in worst-case linear time

We'll now examine a remarkable and theoretically interesting selection algorithm whose running time is $\Theta(n)$ in the worst case. Although the RANDOMIZED-SELECT algorithm from Section 9.2 achieves linear expected time, we saw that its running time in the worst case was quadratic. The selection algorithm presented in this section achieves linear time in the worst case, but it is not nearly as practical as RANDOMIZED-SELECT. It is mostly of theoretical interest.

Like the expected linear-time RANDOMIZED-SELECT, the worst-case linear-time algorithm SELECT finds the desired element by recursively partitioning the input array. Unlike RANDOMIZED-SELECT, however, SELECT *guarantees* a good split by choosing a provably good pivot when partitioning the array. The cleverness in the algorithm is that it finds the pivot recursively. Thus, there are two invocations of SELECT: one to find a good pivot, and a second to recursively find the desired order statistic.

The partitioning algorithm used by SELECT is like the deterministic partitioning algorithm PARTITION from quicksort (see Section 7.1), but modified to take the element to partition around as an additional input parameter. Like PARTITION, the PARTITION-AROUND algorithm returns the index of the pivot. Since it's so similar to PARTITION, the pseudocode for PARTITION-AROUND is omitted.

The SELECT procedure takes as input a subarray $A[p : r]$ of $n = r - p + 1$ elements and an integer i in the range $1 \le i \le n$. It returns the ith smallest element of A. The pseudocode is actually more understandable than it might appear at first.

SELECT(A, p, r, i)